

TinyMouse

Go Skills Roadmap

Everything needed to implement the first real TinyMouse

Learn Go by building TinyMouse, not before building TinyMouse.

TinyMouse is a content-addressed graph of scientific computation. Go implements the infrastructure: the CLI, local cache, container runner, provenance records, lineage DAG, registry integration, and eventually a small API/UI backend. Scientific code itself stays language-agnostic inside OCI containers.

1. What Go Must Do in TinyMouse

```
tinymouse CLI
|
+-- parse commands and manifests
+-- identify Git commit provenance
+-- resolve and cache immutable data artifacts
+-- invoke Docker/Podman
+-- mount inputs read-only
+-- capture logs and exit status
+-- hash and register outputs
+-- persist execution provenance
+-- traverse the scientific DAG
+-- talk to an OCI registry
+-- later: expose local HTTP/API endpoints
```

The first version should be a boring, reliable systems tool. TinyMouse does not need fancy distributed systems machinery before the basic D0 -> container -> D1 lineage is unquestionably correct.

2. Skill Priority Legend

Priority	Meaning	When
P0	Essential to make the first end-to-end TinyMouse work.	Milestone 1
P1	Needed for branching, caching, clean architecture, and a good local developer experience.	Milestone 2
P2	Needed for shared registries, collaboration, stronger security, and larger deployments.	Milestone 3+

3. Core Go Language Skills

Priority	Skill	What to know	TinyMouse use
P0	Packages and modules	go.mod, imports, package visibility, internal packages.	Project structure.
P0	Structs	Model Dataset, Artifact, Execution, ContainerRef, GitRef, Branch.	Domain model.
P0	Methods	Behavior on domain types without inheritance.	Validation and IDs.
P0	Interfaces	Small interfaces such as Runner, Store, Registry, LineageStore.	Replaceable backends.
P0	Errors	Return errors, wrap with %w, errors.Is / errors.As.	Reliable diagnostics.
P0	Slices and maps	Inputs, outputs, parameters, graph adjacency.	Manifests/DAG.
P0	Pointers vs values	Mutation, zero values, copying, receiver choice.	Correct APIs.
P0	JSON/YAML struct tags	Stable serialization and decoding.	Manifests.
P1	Generics - basic only	Use sparingly for helpers; avoid premature abstraction.	Utilities.

4. Filesystem and Content-Addressed Storage

- `os` and `path/filepath`: create directories, stat files, permissions, rename, remove, and walk trees.
- `io.Reader` and `io.Writer`: stream large artifacts rather than loading them into memory.
- `io.Copy` and `io.MultiWriter`: copy and hash a stream in one pass.
- `crypto/sha256` and `encoding/hex`: generate content identities.
- `bufio`: efficient log and metadata I/O.
- Temporary files/directories: stage downloads and outputs before they are committed to the cache.
- Atomic writes: write to a temporary path, `fsync` where appropriate, then rename into place.
- Directory layout design: e.g. `~/tinymouse/blobs/sha256/<digest>`, `manifests/`, `logs/`, `lineage.db`.
- Cache lookup: ensure `_local(digest)` should return immediately when an artifact already exists.
- Read-only semantics: protect registered inputs from accidental mutation by analyses.

```
remote artifact
  |
  | first use
  v
~/tinymouse/blobs/sha256/abc...
  |
  | read-only bind mount
  v
analysis container
```

This is one of the most important TinyMouse-specific Go areas. The local artifact cache is how we avoid re-pulling Allen-sized data for every computation while still keeping data outside the container image.

5. Hashing, Canonicalization, and Deterministic IDs

- SHA-256 hashing of files and small manifests.
- Stable hashing of directories: define deterministic traversal order and include relative paths in the digest.
- Canonical parameter serialization: semantically identical parameter maps must hash identically.
- Avoid hashing mutable names/tags when immutable digests are available.
- Understand that execution identity differs from output identity.
- Design typed IDs such as `tm://artifact/sha256:...`, `tm://execution/sha256:...`, `tm://branch/...`

```
execution_key = H(
    sorted input digests,
    OCI image digest,
    canonical parameters,
    command,
    TinyMouse contract version
)
```

6. CLI Engineering

TinyMouse starts life as a CLI, so command-line ergonomics matter early.

- `os.Args` basics, then a mature CLI library such as `Cobra` when command count grows.
- Subcommands: `init`, `dataset add`, `run`, `inspect`, `lineage`, `branch`, `status`, `push`, `pull`.
- Flags, required arguments, defaults, validation, and mutually exclusive options.
- Exit codes: distinguish user error, execution failure, missing artifact, and internal failure.

- stdout vs stderr discipline so commands are scriptable.
- Human-readable output plus optional --json output for automation.
- Context cancellation: Ctrl-C should cancel a container run cleanly.

7. Running Containers from Go

For the first implementation, TinyMouse can shell out to Docker/Podman safely. A direct Engine API client can come later.

- os/exec: start external processes, pass arguments without shell interpolation, inspect exit codes.
- exec.Cmd.StdoutPipe/StderrPipe or CombinedOutput for logs.
- context.Context with exec.CommandContext for cancellation/timeouts.
- Build safe docker run arguments programmatically.
- Bind-mount cached inputs read-only and a temporary output directory read-write.
- Resolve mutable image tags to an immutable image digest before recording provenance.
- Capture the actual container/image identity used for execution.
- Avoid privileged mode and avoid exposing the Docker socket to analysis containers.

```
docker run --rm --mount type=bind,src=<cached-input>,dst=/input,readonly --mount
type=bind,src=<staging-output>,dst=/output <image@sha256:digest>
```

8. Processes, Signals, and Context

- context.Context: propagate cancellation through runner, downloads, DB operations, and HTTP calls.
- os/signal: handle SIGINT/SIGTERM so interrupted analyses are recorded as failed/cancelled.
- Process exit status and error classification.
- Streaming stdout/stderr while simultaneously writing persistent execution logs.
- Cleanup with defer: temporary directories and partial artifacts must not leak.

9. Concurrency - Enough, Not Everywhere

Go makes concurrency tempting. TinyMouse needs some of it, but correctness of provenance matters more than maximum parallelism.

- Goroutines and channels.
- sync.WaitGroup for joining concurrent work.
- sync.Mutex / RWMutex for local shared state.
- errgroup for related concurrent operations that should fail together.
- Bounded concurrency with semaphores/worker pools.
- Race detector: go test -race.
- Know when not to use concurrency: hashing a single streaming artifact or a simple local DB transaction can remain sequential.

Useful early example: stream container stdout and stderr concurrently while the main goroutine waits for process completion.

10. SQLite and Persistence

- database/sql fundamentals: opening DBs, queries, Exec, QueryRow, Scan.
- Transactions: an execution and its output lineage should be committed atomically.
- Prepared statements and parameterized queries.
- Schema migrations.
- Indexes for digest lookup and parent/child traversal.
- Foreign keys and uniqueness constraints.
- SQLite WAL mode basics for a responsive local tool.
- Store metadata in SQLite; keep large blobs in the content store/object storage.

```
artifacts(id, digest, kind, location, size, created_at)
executions(id, key, image_digest, git_commit, command, status, ...)
execution_inputs(execution_id, artifact_id)
execution_outputs(execution_id, artifact_id)
branches(id, name, head_artifact_id, ...)
```

11. Graph/DAG Skills

- Represent a DAG with IDs and parent/child relationships rather than an in-memory tree.
- Depth-first search (DFS) and breadth-first search (BFS).
- Ancestor and descendant traversal.
- Cycle detection: scientific lineage must remain acyclic for transformation history.
- Topological sorting for ordered lineage views.
- Lowest/common ancestor ideas for comparing where two analyses diverged.
- Deduplication/reconvergence: two paths may point to the same content-addressed artifact.
- Do not introduce a graph database until SQLite traversal becomes an actual bottleneck.

12. Serialization and Manifest Design

- encoding/json deeply enough to control field names, optional fields, custom marshal/unmarshal if needed.
- YAML library usage for human-authored execution manifests.
- Schema version fields such as tinymouse.dev/v1alpha1.
- Strict validation: reject unknown/invalid combinations when correctness matters.
- Canonicalize before hashing; human-friendly YAML is not itself a canonical identity format.
- Backward compatibility and migrations as the manifest evolves.

13. HTTP and OCI Registry Skills

These are mostly P2, after local execution works.

- net/http client basics: requests, headers, status codes, streaming bodies, contexts.
- Authentication patterns: bearer tokens/basic registry auth without leaking credentials.
- OCI concepts: manifests, blobs, descriptors, digests, media types, tags vs digests.
- Push/pull blobs efficiently and skip blobs already present.

- Registry referrers/subject relationships if TinyMouse stores provenance as OCI-associated artifacts.
- Retry only safe/idempotent operations; understand partial upload behavior.
- TLS and certificate validation; never silently disable verification.

14. Git Integration Skills

- Use os/exec with git initially: rev-parse HEAD, remote get-url origin, status --porcelain.
- Detect dirty working trees and record that provenance explicitly.
- Understand commit SHA vs branch name: commits are provenance; branch names are mutable convenience.
- Later, optionally use a native Go Git library if removing the git binary dependency becomes worthwhile.

```
git remote URL + commit SHA + dirty-state
                |
                v
            execution provenance
```

15. Testing Skills

TinyMouse is infrastructure. Tests are part of the product, not decoration.

- Standard testing package and table-driven tests.
- Temporary test directories with t.TempDir().
- Golden tests for manifests and lineage text output.
- Interface fakes for Runner, Store, and Registry.
- Integration tests that execute a tiny fixture container.
- Test deterministic hashing aggressively.
- Test failure paths: missing input, non-zero container exit, interrupted run, corrupted cache entry.
- go test -race for concurrent code.
- Fuzz testing for manifest parsers and canonicalization where useful.

16. Logging and Observability

- Structured logging using slog.
- Stable fields: execution_id, artifact_digest, image_digest, duration, status.
- Separate TinyMouse diagnostic logs from captured scientific container stdout/stderr.
- Timing and basic counters later; do not build an observability platform for v0.1.

17. Security Skills Relevant to TinyMouse

- Threat-model untrusted container execution.
- Understand Unix permissions, bind mounts, read-only filesystems, user IDs, and least privilege.
- Never construct shell command strings from untrusted parameters; pass argv directly.
- Digest verification after downloads.
- Secret handling through environment/file injection without persisting secret values in provenance.

- Path traversal defenses when extracting archives or accepting output paths.
- Resource limits for containers: memory, CPUs, potentially GPU selection.
- Image signing/verification can be added later; understand the concept before implementing it.

18. API / Web Backend Skills for the Later Graph UI

- net/http server fundamentals.
- JSON REST endpoints first; no need for a large framework.
- Routing, middleware, request validation, graceful shutdown.
- Serve lineage queries such as ancestors, children, siblings, branch comparison.
- CORS/auth only when a real remote UI requires them.
- WebSockets/server-sent events only if live execution logs become useful.

19. Go Tooling You Should Be Comfortable With

Tool	Why
go mod init / tidy	Dependency/module management.
go fmt / gofmt	Canonical formatting.
go test ./...	Run the test suite.
go test -race ./...	Detect data races.
go vet ./...	Static checks.
go build	Build the TinyMouse binary.
go install	Install CLI during development.
pprof - later	Investigate performance only when needed.
Delve - optional	Debugger for difficult runtime problems.

20. Suggested Go Package Structure

```

tinymouse/
├── cmd/
│   └── tinymouse/           # main package
├── internal/
│   ├── model/              # Artifact, Execution, Branch, IDs
│   ├── digest/             # SHA-256 + canonicalization
│   ├── cache/              # local content-addressed store
│   ├── runner/             # Runner interface + Docker implementation
│   ├── lineage/            # DAG operations
│   ├── store/              # SQLite persistence
│   ├── gitmeta/            # Git provenance
│   ├── manifest/           # YAML/JSON parsing + validation
│   └── registry/           # OCI integration (later)
└── go.mod

```

Do not create all packages on day one. This is the destination structure. Begin with model, digest, cache, runner, and store.

21. Recommended Learning Sequence - Build the Mouse While Learning

Step	Topic	Learn	TinyMouse checkpoint
------	-------	-------	----------------------

1	Go essentials	structs, methods, errors, modules, slices/maps	Define Artifact and Execution types.
2	Filesystem + SHA-256	io, os, filepath, crypto/sha256	`tinymouse artifact add file.npy` hashes and caches a file.
3	CLI	flags/subcommands/output	`tinymouse inspect <digest>`.
4	Processes	os/exec, context, exit codes	Run `hello-world` or a tiny fixture container.
5	Container contract	bind mounts, staging outputs	D0 -> C1 -> D1 works locally.
6	SQLite	schema, transactions	Persist D0/E1/D1.
7	DAG traversal	DFS/BFS/topological reasoning	`tinymouse lineage D1`.
8	Caching	execution keys and lookup	Identical rerun returns existing D1.
9	Branching	parents/siblings/named heads	D1 -> C2 -> D2 and D1 -> C3 -> D3.
10	Registry/HTTP	OCI + net/http	Share lineage with another machine.

22. Things You Do NOT Need to Learn First

- Kubernetes.
- Microservices architecture.
- gRPC.
- Distributed consensus/Raft.
- A graph database.
- Advanced Go reflection.
- Complex generics.
- A custom container runtime.
- A custom object store.
- A custom OCI registry.
- Event-driven architecture or message queues.

Those can become relevant only after TinyMouse has users or scale. None is necessary to prove the core idea.

23. The Minimum Go Competence Needed to Start Coding

You do not need to finish this roadmap before implementation. Start when you can comfortably do these:

- Create a Go module and multiple packages.
- Define structs and methods.
- Return and wrap errors.
- Read/write files.
- Compute SHA-256.
- Serialize a struct to JSON.
- Run an external process with os/exec.
- Write a table-driven unit test.

At that point, Milestone 1 becomes the course. Every missing Go concept should be learned immediately before TinyMouse needs it.

24. First TinyMouse Programming Target

The first slice should deliberately ignore OCI registries, collaboration, web UI, and remote datasets.

```
$ tinymouse init

$ tinymouse artifact add ./input.txt
artifact sha256:AAA

$ tinymouse run      --input sha256:AAA      --image tiny-analysis@sha256:BBB

execution sha256:EEE
output      sha256:CCC

$ tinymouse lineage sha256:CCC

sha256:AAA
  |
  v
execution sha256:EEE
  |
  v
sha256:CCC
```

When this works with immutable inputs, a read-only mount, an output hash, and a persisted execution record, the conceptual heart of TinyMouse is alive.

25. TinyMouse Go Mastery Checklist

- ☐ I can explain `io.Reader/io.Writer` and use streaming APIs.
- ☐ I can hash a multi-GB file without loading it into memory.
- ☐ I can make a cache write atomic.
- ☐ I can model TinyMouse domain objects with structs.
- ☐ I can design a small Go interface and implement/fake it.
- ☐ I can invoke Docker safely without a shell.
- ☐ I can cancel an execution with `context.Context`.
- ☐ I can distinguish an image tag from an image digest.
- ☐ I can use database/sql with a transaction.
- ☐ I can traverse ancestors/descendants in a DAG.
- ☐ I can generate a deterministic execution key.
- ☐ I can write integration tests for an actual container run.
- ☐ I can stream logs without corrupting provenance state.
- ☐ I can explain why data stays outside container layers.
- ☐ I can explain exactly which facts TinyMouse must persist for reproducibility.

26. The One-Sentence Curriculum

Learn enough Go to turn immutable data + an immutable container into a new immutable artifact, record that edge correctly, and then keep extending the graph.